LLM EVALUATION & BENCHMARKING PLATFORM

Software Requirements Specification

(SRS) — IEEE 830 style

Document Version 1.0
September 2026

Prepared by
Monish D — 24BDS1033
Soaham S — 24BDS1021

Project: LLM Evaluation and Benchmarking Platform
Course: Software Engineering — Digital Assignment

Table of Contents

Table of Contents 2
1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions, Acronyms and Abbreviations 3
1.4 Overall Description 3
2. Functional Requirements 3
2.1 Model & Provider Management 3
2.2 Dataset & Prompt Management 4
2.3 Benchmark Execution 4
2.4 Quality Evaluation 4
2.5 Hallucination & Reliability Evaluation 4
2.6 Capability Profiling & Industry Comparison (Core Feature) 5
2.7 Efficiency & Cost Evaluation 5
2.8 Reporting & Visualization 5
3. Non-Functional Requirements 6
3.1 Performance 6
3.2 Scalability & Extensibility 6
3.3 Reliability & Fault Tolerance 6
3.4 Security 6
3.5 Usability 6
3.6 Maintainability 6
3.7 Cost Constraint 6
4. External Interface Requirements 6
4.1 User Interfaces 7
4.2 Software Interfaces 7
4.3 Communication Interfaces 7
5. Data Requirements 7
6. Traceability 7

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) translates the User Requirements Specification (URS) into precise, verifiable functional and non-functional requirements for the LLM Evaluation and Benchmarking Platform. It is written for the development team (backend, evaluation-engine, and frontend engineers) and is the contractual basis for design, implementation, and testing.

1.2 Scope

The system under specification evaluates multiple LLMs against standardized and user-supplied datasets, computes quality, reliability/hallucination, capability, efficiency, and cost metrics, and produces a comparative positioning of each evaluated model against a maintained set of industry-standard baseline models, broken down by capability category (coding, mathematics, reasoning, creative writing, summarization, factual/grounded QA, long-context/out-of-context reasoning). The system is delivered as a web application with a Python backend and a React frontend, deployable on free-tier cloud infrastructure.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SUT	System/Model Under Test
Baseline Set	A maintained registry of industry-standard reference models with pre-computed or periodically refreshed benchmark scores
Capability Category	One of: Coding, Mathematics, Reasoning, Creative Writing, Summarization, Factual/Grounded QA, Long-Context / Out-of-Context Reasoning, Instruction Following
Tier	A qualitative bucket (Frontier / Strong / Mid / Weak) derived from a model's percentile within a capability category
Adapter	A provider-specific integration module implementing a common LLM-call interface

1.4 Overall Description

The platform is organized as: Frontend (React) → Backend API (FastAPI) → Benchmark Manager → LLM Integration Layer (adapters) → Evaluation Engine (quality, hallucination, capability, efficiency, cost sub-modules) → Industry Comparison Engine (new, project-defining module) → Database → Reporting Layer. The Industry Comparison Engine is what differentiates this platform from a plain benchmark runner: it normalizes a model's raw scores against the Baseline Set and emits tiered, per-category positioning.

2. Functional Requirements

2.1 Model & Provider Management

ID	Requirement
FR-1.1	The system shall allow adding/removing LLM provider credentials (API keys) through a secure settings interface.

ID	Requirement
FR-1.2	The system shall support at minimum: OpenAI-compatible APIs, OpenRouter, Anthropic, Google Gemini, and self-hosted/open-source models via a common adapter interface.
FR-1.3	The system shall allow selecting 1..N models for a single benchmark run.
FR-1.4	The system shall record model name, version/snapshot id, and provider for every run.

2.2 Dataset & Prompt Management

ID	Requirement
FR-2.1	The system shall ship with prebuilt benchmark datasets covering: coding, mathematics, logical/commonsense reasoning, summarization, creative writing, factual/grounded QA, and long-context comprehension.
FR-2.2	The system shall allow a user to upload a custom dataset (JSON/CSV) containing prompts and optional reference answers.
FR-2.3	The system shall validate uploaded datasets against a defined schema and reject malformed files with a clear error.
FR-2.4	The system shall allow configuring shared model parameters (temperature, max_tokens, system prompt) applied identically across all selected models for a run.

2.3 Benchmark Execution

ID	Requirement
FR-3.1	The system shall execute every selected task against every selected model using identical prompts and parameters.
FR-3.2	The system shall record, per request: raw response, latency, input tokens, output tokens, and any error/failure code.
FR-3.3	The system shall continue executing remaining tasks/models if an individual request fails (fault isolation), and shall mark the failed cell as "failed" rather than omitting it.
FR-3.4	The system shall support batch/asynchronous execution and report run progress to the frontend.
FR-3.5	The system shall persist the full run configuration for reproducibility.

2.4 Quality Evaluation

ID	Requirement
FR-4.1	The system shall compute accuracy for tasks with reference answers using exact-match, F1, or semantic-similarity as appropriate to the task type.
FR-4.2	The system shall compute relevance, completeness, and instruction-following scores via LLM-as-a-Judge for open-ended tasks, using a documented judge model and rubric.
FR-4.3	The system shall record which evaluation method produced each score.

2.5 Hallucination & Reliability Evaluation

ID	Requirement
FR-5.1	The system shall compute a response-level hallucination rate: proportion of responses containing at least one fabricated/unsupported/incorrect claim.
FR-5.2	The system shall, where a reference or source document exists, compute a claim-level hallucination rate via claim extraction and claim verification.
FR-5.3	The system shall compute a groundedness score for tasks that supply source context (e.g., RAG-style QA).
FR-5.4	The system shall allow a user to inspect each flagged claim together with the evidence used to flag it.
FR-5.5	The system shall compute a consistency score by re-running a sample of prompts and measuring output stability.

2.6 Capability Profiling & Industry Comparison (Core Feature)

ID	Requirement
FR-6.1	The system shall maintain a Baseline Set of industry-standard models with pre-computed scores per capability category, refreshable on a schedule.
FR-6.2	The system shall map every benchmark task to exactly one capability category (Coding, Mathematics, Reasoning, Creative Writing, Summarization, Factual/Grounded QA, Long-Context/Out-of-Context Reasoning, Instruction Following).
FR-6.3	The system shall compute, per model per category, a normalized score (e.g., z-score or percentile) relative to the Baseline Set.
FR-6.4	The system shall classify each model-category pair into a Tier (Frontier / Strong / Mid / Weak) using documented, adjustable thresholds.
FR-6.5	The system shall generate a capability profile (radar chart data) per model across all categories.
FR-6.6	The system shall generate a plain-language positioning summary per model (e.g., "Frontier-tier on coding; Weak-tier on long-context reasoning versus current baselines").
FR-6.7	The system shall label every baseline comparison with the baseline model's version and the date its reference score was captured.

2.7 Efficiency & Cost Evaluation

ID	Requirement
FR-7.1	The system shall record per-request latency (time-to-first-token and total time, where the provider exposes it).
FR-7.2	The system shall record input/output token counts per request.
FR-7.3	The system shall compute estimated cost per request using a configurable, per-provider pricing table.
FR-7.4	The system shall compute cost-per-successful-task and a cost-vs-performance metric for comparative plotting.

2.8 Reporting & Visualization

ID	Requirement
FR-8.1	The system shall present a comparative dashboard with tables and charts covering all metric families.
FR-8.2	The system shall let a user drill into an individual response with its scores and rationale.
FR-8.3	The system shall export a run report as PDF and the raw results as CSV/JSON.
FR-8.4	The system shall retain benchmark run history and allow re-viewing a past run.

3. Non-Functional Requirements

3.1 Performance

The system shall support batch execution of at least 50 concurrent evaluation tasks without failure on the target free-tier infrastructure, using asynchronous request handling and rate-limit-aware throttling per provider.

3.2 Scalability & Extensibility

Adding a new LLM provider shall require only a new adapter module conforming to the common interface, with no changes to the Evaluation Engine or Industry Comparison Engine. Adding a new capability category or dataset shall not require backend redeployment beyond a configuration/migration change.

3.3 Reliability & Fault Tolerance

A failure in one provider/model shall not abort the overall benchmark run (FR-3.3). All partial results shall be persisted incrementally, not only at run completion.

3.4 Security

API keys shall be stored encrypted at rest and never returned to the frontend or embedded in exported reports. All external API calls shall occur server-side only.

3.5 Usability

A first-time user shall be able to configure and launch a benchmark run without reading external documentation, guided by in-app defaults (a preselected dataset, model list, and parameters).

3.6 Maintainability

The codebase shall separate adapters, evaluation metrics, the Industry Comparison Engine, and UI components into independently testable modules with unit test coverage for each evaluation metric.

3.7 Cost Constraint

The production deployment shall run entirely on free tiers: free hosting (e.g., Railway free tier / Render / Fly.io free tier), free database (e.g., Supabase free tier / Neon free tier), free or free-credit inference (e.g., OpenRouter free models, NVIDIA NIM free credits, Cloudflare Workers AI free allocation, self-hosted small models). This constraint shall be explicit in every design decision.

4. External Interface Requirements

4.1 User Interfaces

A responsive web UI (React) providing: model/dataset selection, run configuration, live run progress, results dashboard, model detail/drill-down, capability radar view, and report export.

4.2 Software Interfaces

- REST API (FastAPI/Flask) between frontend and backend, JSON payloads.
- Outbound HTTPS calls to LLM provider APIs via provider adapters.
- PostgreSQL (or SQLite in development) for persistence, accessed via an ORM.
- Optional object storage for exported PDF/CSV reports.

4.3 Communication Interfaces

All communication uses HTTPS/TLS. Long-running benchmark runs shall expose a polling or WebSocket-based progress endpoint.

5. Data Requirements

Core entities: Provider, Model, Dataset, Task, BenchmarkRun, RunResult (per model per task), EvaluationScore (per RunResult per metric), BaselineModel, BaselineScore (per category), CapabilityCategory, and Report. A RunResult links to one or more EvaluationScore rows (quality, hallucination, capability tier, efficiency, cost), and each model's CapabilityProfile is derived by aggregating EvaluationScore rows against BaselineScore per CapabilityCategory.

6. Traceability

Each functional requirement in this SRS traces back to one or more URS user stories (e.g., FR-6.1–FR-6.7 trace to UR-05–UR-09, the industry-positioning requirements that define this platform's novelty relative to plain benchmark runners such as generic leaderboards).